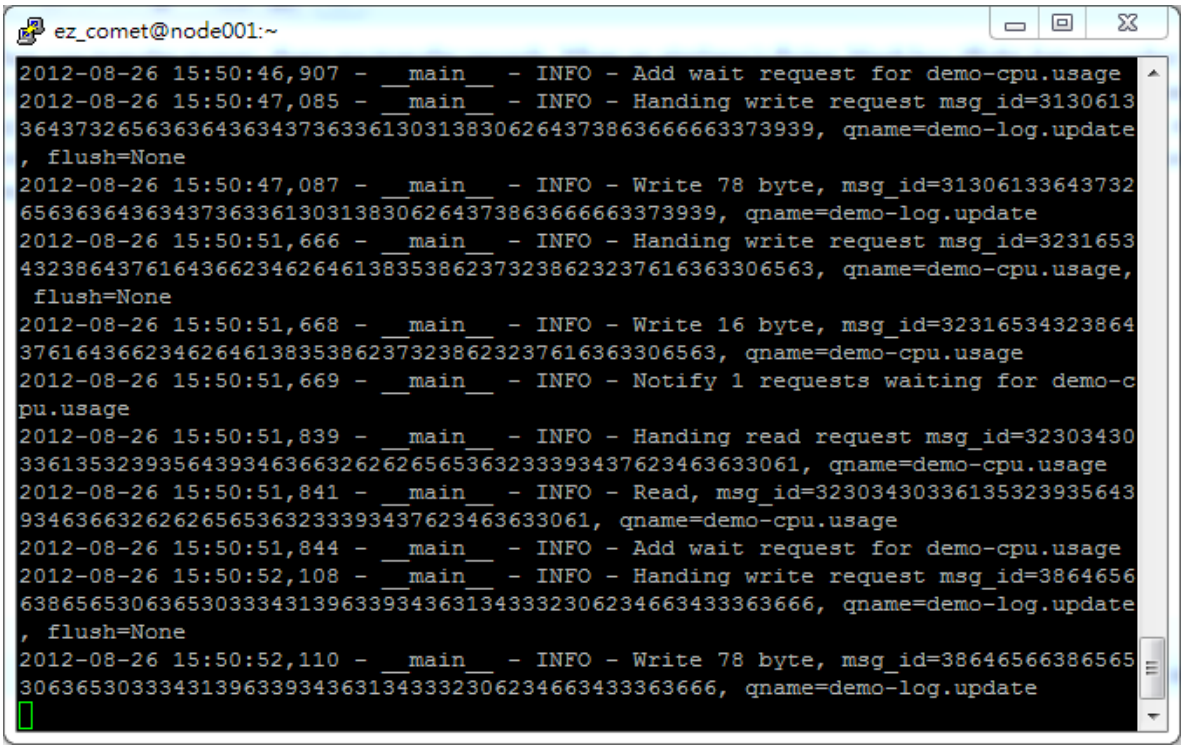
Good logging practice in Python

Aug 26, 2012

14 minute read

Logging is important in our life. When you transfer money, there will be logs for the transaction. When an airplane is flying, the black box will record flight data. If something goes wrong, people can read the log and get a chance to figure out what happened. Likewise, logging is also essential for software development. When a program crashes, if there is no logging record, you have little chance to understand what happened. Now, let's see an example in real life. The following screenshot is the log data from one of my running server:



Not only for the servers, but logging is also crucial for almost any kind of software systems. For instance, say you have a GUI app, and it keeps writing logging records in a text file. With that, when your program crashes on the customer's PC, you can ask

them to send the log file to you, then you might be able to figure out why. In the past, I build different apps, all with good logging practices, and this solves countless problems for me. Trust me, you will never know what kind of strange issues there will be in different environments. I once received an error log report like this:

```
2011-08-22 17:52:54,828 - root - ERROR - [Errno 10104] getaddrinfo f
Traceback (most recent call last):
  File "<string>", line 124, in main
  File "<string>", line 20, in __init__
  File "h:\workspace\project\build\pyi.win32\mrdj\outPYZ1.pyz/wx._co
  File "h:\workspace\project\build\pyi.win32\mrdj\outPYZ1.pyz/wx._co
  File "<string>", line 84, in OnInit
  File "h:\workspace\project\build\pyi.win32\mrdj\outPYZ1.pyz/twiste
  File "h:\workspace\project\build\pyi.win32\mrdj\outPYZ1.pyz/twiste
  File "h:\workspace\project\build\pyi.win32\mrdj\outPYZ1.pyz/twiste
  File "h:\workspace\project\build\pyi.win32\mrdj\outPYZ1.pyz/twiste
  File "h:\workspace\project\build\pyi.win32\mrdj\outPYZ1.pyz/twiste
  File "h:\workspace\project\build\pyi.win32\mrdj\outPYZ1.pyz/socket
gaierror: [Errno 10104] getaddrinfo failed
```

And eventually, by looking into the log file, I figured out that the customer PC is infected by a virus which makes the `gethostname` function call failed. See, how can you even know that if there is no log to read?

print is not a good idea

Although logging is important, not all developers know how to do them correctly. Some people like to insert print statements while developing and remove those statements when they are not needed anymore. Their code may look like this:

```
print('Start reading database')
records = model.read_recrods()
print('# records', records)
print('Updating record ...')
model.update_records(records)
print('done')
```

It works fine when the program is a simple small script, but for a complex system, you better not to use this approach. First of all, you cannot display only the important messages in the log. You may see lots of garbage messages and can't find anything useful. You also cannot control those print statements without modifying code, you may forget to remove those unused prints. And as all the printed messages go into stdout, which is terrible when you have data to output to stdout. Of course you can print messages to stderr instead, but still, it is not a good practice to use print for logging.

Use Python standard logging module

So, how do you do logging correctly? It's easy, use the standard Python logging module. Thanks to the Python community, logging is a standard module, it was well

designed to be easy-to-use and very flexible. You can use the logging system like this:

```
import logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

logger.info('Start reading database')
# read database here
records = {'john': 55, 'tom': 66}
logger.debug('Records: %s', records)
logger.info('Updating records ...')
# update records here
logger.info('Finish updating records')
```

You can run it and see

```
INFO:__main__:Start reading database
INFO:__main__:Updating records ...
INFO:__main__:Finish updating records
```

What's different between the print approach you asked. Well, of course, there are benefits:

- You can control message level and filter out not important ones
- You can decide where and how to output later

There are different importance levels you can use, `DEBUG`, `INFO`, `WARNING`, `ERROR`, and `CRITICAL`. By giving different level to the logger and handler, you can write only error messages to a specific log file, or record debug details when debugging. Let's change the logger level to `DEBUG` and see the output again

```
logging.basicConfig(level=logging.DEBUG)
```

The output:

```
INFO:__main__:Start reading database
DEBUG:__main__:Records: {'john': 55, 'tom': 66}
INFO:__main__:Updating records ...
INFO:__main__:Finish updating records
```

As you can see, the debugging records now appear in the output. Like we mentioned previously, besides changing the log level, you can also decide how to process these messages. For example, say you want to write the logs to a file, you can then use the `FileHandler` like this:

```
import logging

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)

# create a file handler
```

```
handler = logging.FileHandler('hello.log')
handler.setLevel(logging.INFO)

# create a logging format
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)
handler.setFormatter(formatter)

# add the file handler to the logger
logger.addHandler(handler)

logger.info('Hello baby')
```

The `FileHandler` is just one of many useful build-in handlers. In Python, there is even an SMTP log hander for sending records to your mailbox or one for sending the logs to an HTTP server. Cannot find a handler for your need? No worries, you can also write your own custom logging handler if you want. For more details, please reference official documents: Basic Tutorial, Advanced Tutorial and Logging Cookbook.

Write logging records everywhere with a proper level

With the flexibility of the logging module, it's a good practice to write logging record everywhere with appropriate level and configure them later. What is the appropriate level to use, you may ask. Let me share my experience here.

In most cases, you don't want to flood your log file with too many trivial information. And as the name implies, the `DEBUG` level is for debugging. Therefore, the `DEBUG` level should only be enabled when you are debugging. I use the `DEBUG` level only for logging verbose details. Especially when the data is huge, or the frequency is high, such as records of algorithm internal state changes in a for-loop.

```
def complex_algorithm(items):
    for i, item in enumerate(items):
        # do some complex algorithm computation
        logger.debug('%s iteration, item=%s', i, item)
```

I use `INFO` level for routines. For example, handling requests or server state changed.

```
def handle_request(request):
    logger.info('Handling request %s', request)
    # handle request here
    result = 'result'
    logger.info('Return result: %s', result)

def start_service():
    logger.info('Starting service at port %s ...', port)
    service.start()
    logger.info('Service is started')
```

I use `WARNING` when something needs your attention, but not an error. For example, when a user attempts to log in with a wrong password, or, the connection is slow.

```
def authenticate(user_name, password, ip_address):
    if user_name != USER_NAME and password != PASSWORD:
        logger.warning('Login attempt to %s from IP %s', user_name,
        return False
    # do authentication here
```

I use `ERROR` level when something is wrong. For example, an exception is thrown, IO operation failure or connectivity issue.

```
def get_user_by_id(user_id):
    user = db.read_user(user_id)
    if user is None:
        logger.error('Cannot find user with user_id=%s', user_id)
        return user
    return user
```

I never use `CRITICAL`, you can use it when something horrible happens. For example, out of memory, the disk is full or a nuclear meltdown (Hopefully that will not actually happen 😅).

Use __name__ as the logger name

It is not a requirement to set the logger name as `__name__`, but by doing that, it brings us some benefits. In case if you don't know, the variable `__name__` is the current module name in Python. For example, you write this in a module `foo.bar.my_module`:

```
logger.getLogger(__name__)
```

It's actually the same as this:

```
logger.getLogger("foo.bar.my_module")
```

Since the Python's logging configuration system follows a hierarchy design, the levels in the hierarchy are separated by dots, just like Python's package and module names. Given `foo.bar.my_module` as the logger name, the hierarchy will be:

```
+ foo
  + bar
    - my_module
```

Say we have many logger names like these

- foo.bar.my_module
- foo.bar.my_module2
- foo.eggs.spam
- foo.eggs.spam2

Then the corresponding hierarchy will be:

```
+ foo
  + bar
    - my_module
    - my_module2
  + eggs
    - spam
    - spam2
```

With that, say you want to configure the loggers for all modules under a package, it can be done easily by configuring the logger with the package as the name. For example, you notice that the log messages from the modules in the `foo.eggs` package are too verbose, you want to filter most of them out. With the configuration hierarchy, you can configure `foo.eggs` package to a higher logging level, say `WARNING`. In this way, for the logs messages from

- foo.eggs.spam
- foo.eggs.spam2

as long as their logging level is not equal or higher than `WARNING`, they will not be processed by the handler.

Besides you can control the logger for the whole package, by using `__name__` as the logger name you can also configure the formatter to output the logger name. It's very helpful to know which module is writing the log.

Capture exceptions with the traceback

While it's a good practice to log a message when something goes wrong, but it won't be helpful if there is no traceback. You should capture exceptions and record them with traceback. Following is an example:

```
try:
    open('/path/to/does/not/exist', 'rb')
except (SystemExit, KeyboardInterrupt):
    raise
except Exception as exception:
    logger.error('Failed to open file', exc_info=True)
```

By calling logger methods with `exc_info=True` parameter, the traceback will be dumped to the logger. As you can see the result

```
ERROR:__main__:Failed to open file
Traceback (most recent call last):
  File "example.py", line 6, in <module>
    open('/path/to/does/not/exist', 'rb')
IOError: [Errno 2] No such file or directory: '/path/to/does/not/exi
```

Since Python 3.5, you can also pass the exception instance to `exc_info` parameter. Besides `exc_info` parameter, you can also call `logger.exception(msg, *args)`, it is the same as calling to `logger.error(msg, *args, exc_info=True)`.

Do not get logger at the module level unless disable_existing_loggers is False

You can see lots of examples out there are getting the loggers at the module level, including this article. I did it just for making the example simple. While this looks harmless, but in fact, there is a pitfall. The problem is, Python logging module by default respects all existing logger before you load the configuration from a file if you get logger at the module level like this

my_module.py:

```
import logging

logger = logging.getLogger(__name__)

def foo():
    logger.info('Hi, foo')

class Bar(object):
    def bar(self):
        logger.info('Hi, bar')
```

main.py

```
import logging

# load my module
import my_module

# load the logging configuration
logging.config.fileConfig('logging.ini')

my_module.foo()
bar = my_module.Bar()
bar.bar()
```

logging.ini

```
[loggers]
keys=root

[handlers]
keys=consoleHandler

[formatters]
keys=simpleFormatter

[logger_root]
level=DEBUG
handlers=consoleHandler
```

```
[handler_consoleHandler]
class=StreamHandler
level=DEBUG
formatter=simpleFormatter
args=(sys.stdout,)

[formatter_simpleFormatter]
format=%(asctime)s - %(name)s - %(levelname)s - %(message)s
datefmt=
```

And you expect to see the records appear in the log, but you will see nothing. Why? Because you create the logger at the module level, you then import the module before loading the logging configuration from a file. The `logging.fileConfig` and `logging.dictConfig` function's default behavior is to disable existing loggers when they are called. So, those setting in the file will not be applied to your logger in `my_module.py`. It's better to get the logger when you need it to avoid the pitfall. It's cheap to create or get a logger. You can write the code like this:

```
import logging

def foo():
    logger = logging.getLogger(__name__)
    logger.info('Hi, foo')

class Bar(object):
    def __init__(self, logger=None):
        self.logger = logger or logging.getLogger(__name__)

    def bar(self):
        self.logger.info('Hi, bar')
```

By doing that, the loggers will be created after you load the configuration. Now the setting will be applied correctly.

Other than avoiding create logger at the module level, since Python 2.7, a new argument `disable_existing_loggers` was added to the `logging.fileConfig` function, and `disable_existing_loggers` key is also added in the schema for`logging.dictConfig` function. By setting it to False, the problem mentioned above can be solved. For example:

```
import logging
import logging.config

logger = logging.getLogger(__name__)

# load config from file
# logging.config.fileConfig('logging.ini', disable_existing_loggers=
# or, for dictConfig
logging.config.dictConfig({
    'version': 1,
    'disable_existing_loggers': False,  # this fixes the problem
    'formatters': {
        'standard': {
            'format': '%(asctime)s [%(levelname)s] %(name)s: %(messa
        },
```

```
    },
    'handlers': {
        'default': {
            'level':'INFO',
            'class':'logging.StreamHandler',
        },
    },
    'loggers': {
        '': {
            'handlers': ['default'],
            'level': 'INFO',
            'propagate': True
        }
    }
})

logger.info('It works!')
```

Use JSON or YAML logging configuration

You can configure your logging system in Python code, but then you need to modify your code whenever you want to change the log configuration. Another way to do it is to use a logging configuration file. After Python 2.7, you can load logging configuration from a dict. It means you can load the logging configuration from a JSON or YAML file. Although you can use the old .ini style logging configuration, it is difficult to read and write. Here let me show you the logging configuration examples in JSON and YAML

logging.json

```
{
  "version": 1,
  "disable_existing_loggers": false,
  "formatters": {
    "simple": {
      "format": "%(asctime)s - %(name)s - %(levelname)s - %(message)
    }
  },

  "handlers": {
    "console": {
      "class": "logging.StreamHandler",
      "level": "DEBUG",
      "formatter": "simple",
      "stream": "ext://sys.stdout"
    },

    "info_file_handler": {
      "class": "logging.handlers.RotatingFileHandler",
      "level": "INFO",
      "formatter": "simple",
      "filename": "info.log",
      "maxBytes": 10485760,
      "backupCount": 20,
      "encoding": "utf8"
    },

    "error_file_handler": {
```

```
      "class": "logging.handlers.RotatingFileHandler",
      "level": "ERROR",
      "formatter": "simple",
      "filename": "errors.log",
      "maxBytes": 10485760,
      "backupCount": 20,
      "encoding": "utf8"
    }
  },

  "loggers": {
    "my_module": {
      "level": "ERROR",
      "handlers": ["console"],
      "propagate": false
    }
  },

  "root": {
    "level": "INFO",
    "handlers": ["console", "info_file_handler", "error_file_handler
  }
}
```

logging.yaml

```
---
version: 1
disable_existing_loggers: False
formatters:
  simple:
    format: "%(asctime)s - %(name)s - %(levelname)s - %(message)s"

handlers:
  console:
    class: logging.StreamHandler
    level: DEBUG
    formatter: simple
    stream: ext://sys.stdout

  info_file_handler:
    class: logging.handlers.RotatingFileHandler
    level: INFO
    formatter: simple
    filename: info.log
    maxBytes: 10485760 # 10MB
    backupCount: 20
    encoding: utf8

  error_file_handler:
    class: logging.handlers.RotatingFileHandler
    level: ERROR
    formatter: simple
    filename: errors.log
    maxBytes: 10485760 # 10MB
    backupCount: 20
    encoding: utf8

loggers:
  my_module:
    level: ERROR
    handlers: [console]
```

```
    propagate: no

root:
  level: INFO
  handlers: [console, info_file_handler, error_file_handler]
```

The following recipe shows you how to read logging configuration from a JSON file:

```
import os
import json
import logging.config

def setup_logging(
    default_path='logging.json',
    default_level=logging.INFO,
    env_key='LOG_CFG'
):
    """Setup logging configuration

    """
    path = default_path
    value = os.getenv(env_key, None)
    if value:
        path = value
    if os.path.exists(path):
        with open(path, 'rt') as f:
            config = json.load(f)
        logging.config.dictConfig(config)
    else:
        logging.basicConfig(level=default_level)
```

One advantage of using JSON configuration is that Python has json as a standard library, unlike YAML, you don't need to install a third-party library. But personally, I prefer YAML. It's clear to read and easier to write. You can install PyYAML and load the YAML configuration with the following recipe:

```
import os
import logging.config

import yaml

def setup_logging(
    default_path='logging.yaml',
    default_level=logging.INFO,
    env_key='LOG_CFG'
):
    """Setup logging configuration

    """
    path = default_path
    value = os.getenv(env_key, None)
    if value:
        path = value
    if os.path.exists(path):
        with open(path, 'rt') as f:
            config = yaml.safe_load(f.read())
        logging.config.dictConfig(config)
    else:
        logging.basicConfig(level=default_level)
```

Now, to setup logging, you can call `setup_logging` when starting your program. It reads `logging.json` or `logging.yaml` by default. You can also set `LOG_CFG` environment variable to load the logging configuration from a path. For example,

```
LOG_CFG=my_logging.json python my_server.py
```

or if you prefer YAML

```
LOG_CFG=my_logging.yaml python my_server.py
```

Use rotating file handler

If you use `FileHandler` for writing logs, the size of the log file will grow with time. Someday, it will occupy all of your disk space. To avoid that situation, you should use `RotatingFileHandler` instead of `FileHandler` in the production environment.

Setup a central log server when you have multiple servers

When you have multiple servers and different log files. You can set up a central log system to collect all vital log messages. Then you can monitor it easily and notice what's wrong in your system.

Conclusions

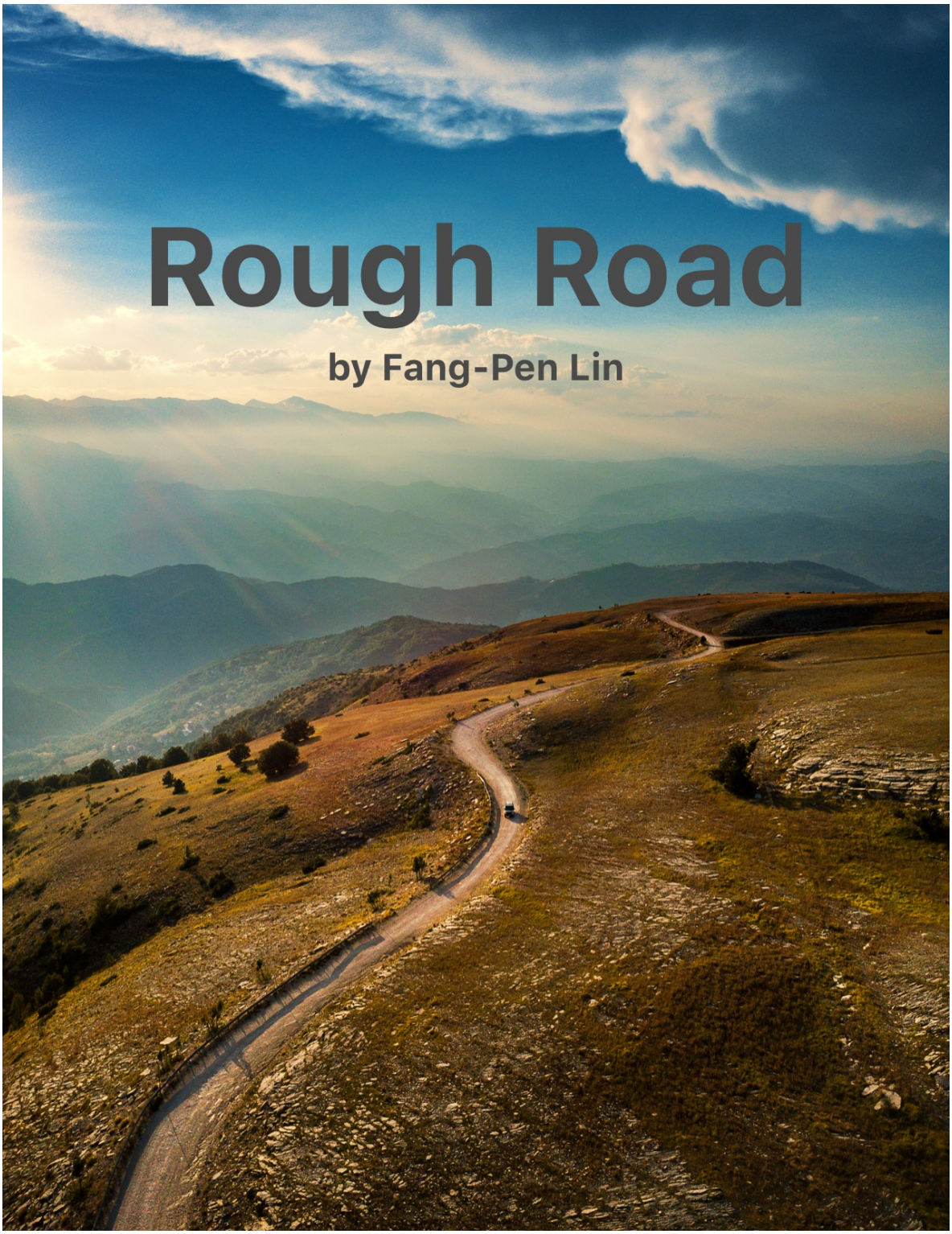
Personally, I think the best part of Python's logging library is that it is a standard library so that you don't have to choose. Remember the Zen of Python?

> *There should be one– and preferably only one –obvious way to do it.*

I'm also happy with its excellent design. It is flexible, you can write your own handlers and filters. There are also third-party handlers such as ZeroMQ logging handler provided by pyzmq, it allows you to send logging messages through a zmq socket.

Finally, if you didn't know how to use the logging system correctly, hopefully, my article could be helpful. With proper logging practice, you can find issues in your system easier. It's an excellent investment, don't you buy it? 😎

Also check out my new book here 👇 :



Rough Road - a zero to master software engineer guide book

You know you want to be a **topnotch** software engineer, but you know it's **never easy** to get there. From time to time, you feel **lost** and don't know **what to do**. This is what you are looking for, a guide from **ground up** to **top level** based on **twenty years intensive programming experience**.

Time limited offer expires on Sep 30 2019, original price ~~$11.99~~, now only **$4.99** via the link below:

Buy it now

Share this post!

Recent articles:

Elliptic Curve Cryptography Explained

Ninja Speed Vim-like Debugging with Ergonomic Keyboard and Trackpad

How I learn programming by building an MMORPG from the ground up 7 times

44 Comments **Fang-Pen's coding note** **Privacy Policy**

Login

Recommend 58 Tweet Share **Sort by Best**

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name

Pascal van Kooten • 4 years ago

Thanks! I love how that yaml reads. Any lessons learned since then (2012) ^_^?

6 • Reply • Share ›

S Charlesworth • 3 years ago

Super useful, thanks man.

5 • Reply • Share ›

Avraham Melamed • 4 years ago

Thanks. Do you know whether it is possible to utilize (for example) time format (e.g. %(asctime)s) in the Filename element ... For example:
info_file_handler:
class: logging.handlers.RotatingFileHandler
level: INFO
formatter: simple
filename: info%(asctime)s.log
maxBytes: 10485760 # 10MB
backupCount: 20
encoding: utf8

The intention is to have the time of file creation reflected in its

The intention is to have the time of file creation reflected in its name.
Of course, it would make sense to use 'w' as the file creation mode...
Thanks for a great article

2 • Reply • Share ›

Esteban Echeverry Perez • 6 months ago

Thanks a lot! Your advice about not instantiating the logger objects at the module level is priceless! Always thought there was something wrong with such practice...

• Reply • Share ›

Gonzalo Amadio • a year ago

Nice! , When you say this "With the configuration hierarchy, you can configure foo.eggs package to a higher logging level, say WARNING" , how do you achieve that?

• Reply • Share ›

Hank Birkdale • a year ago

Really good article. Question for you regarding central logging. For a large project with lots of modules, does it make sense to have a global constant name variable that you set in the main and then have all the modules check against? Something like:
LOGGERNAME = __name__ # say in your main
and then in all the modules
logger = logging.getLogger(LOGGERNAME)
or is there a python best practice for this?

• Reply • Share ›

Michał Zawadzki • a year ago

Is there any way to set the extension of the rotating logs to .log, or do I need a separate python script to do it?

• Reply • Share ›

Алексей Зосимов • 2 years ago

Thanks!!!

• Reply • Share ›

Knight • 2 years ago

How can we share a Rotating file handler across all python files. Can you share the code for it

• Reply • Share ›

Pablo deGaffe • 2 years ago • edited

In your second code block under the section **Use Python standard logging module** you have:

```
....
logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)

# create a file handler
handler = logging.FileHandler('hello.log')
handler.setLevel(logging.INFO)
....
```

Why do you have `logger.setLevel(logging.INFO)` *and* `handler.setLevel(logging.INFO)` in your sample code? Don't you only need one of those to set logging.INFO? And I assume the only one you would need would be `logger.setLevel(logging.INFO)` and the other can be removed, is that correct?

Reply • Share ›

Fang-Pen Lin Mod → Pablo deGaffe • 2 years ago

Yep, that's correct. In my example, to make it works, you don't need to set a logging level on handler, as logger will filter out INFO and above. However, it's a good practice to set a level on the handler, because you usually have many logger, not just one as INFO. You may have some logger comes with DEBUG level, for those logger you may only want to write to console instead of file. So it's always a good idea to set level to handler as well.

Reply • Share ›

小破 • 2 years ago

thank you very much

Reply • Share ›

Andrey Zloy • 2 years ago

Nice article! Thank you!

Reply • Share ›

sothye • 2 years ago

Indeed, a very useful article !

Reply • Share ›

Jeff - Mci • 3 years ago

when using __name__ as the 'getLogger' name.... shouldn't I see the NAME of the module in the logs instead of "__main__" ?

Reply • Share ›

None → Jeff - Mci • 2 years ago

__name__ is __main__ when you're executing that script at the root level.

Reply • Share ›

Jeff - Mci ➦ None • 2 years ago

thanks

Reply • Share ›

Pavel Duda • 3 years ago

Great overview, I was struggling to get it working as I need using just the "standard documentation" so I was using my own logging class for a long time...

Reply • Share ›

finnich • 3 years ago

Why I am not passing an instance of logger to the setup_logger() method? Should I create a single instance and then run setup_logger()?

Reply • Share ›

Fang-Pen Lin Mod ➦ finnich • 2 years ago

The reason why you don't need to pass logger instance to setup_logging, is because Python logging module is looking at logger's name and configure them globally. So it will affect all the logger under the same package.

Reply • Share ›

Fauzan Elka • 3 years ago

Thanks!

Reply • Share ›

Fauzan Elka ➦ Fauzan Elka • 3 years ago

good tutorial

Reply • Share ›

Victory Chang • 3 years ago

Thank you very much for sharing, I also prefer the way YAML reads and I am glad I came across this article.

Reply • Share ›

aoishi • 3 years ago

Thanks! This post is very useful fo me!

Reply • Share ›

Anu • 3 years ago

cool

Reply • Share ›

Lino Bossio Pittini • 3 years ago

excelent, very grateful for it.

Reply • Share ›

Diego Quintana Valenzuela • 3 years ago

Hi, thanks for the tutorial. Would you mind to explain the fields in the config.yaml (or json for this matter) and an example of when should I call this method? 'when running my program' means that should be in the __init__.py file of the module? Why I am not passing an instance of logger to the setup_logger() method? Should I create a single instance and then run setup_logger()?

Reply • Share ›

Fang-Pen Lin Mod → Diego Quintana Valenzuela • 2 years ago

The reason why you don't need to pass logger instance to setup_logging, is because Python logging module is looking at logger's name and configure them globally.

Reply • Share ›

Jinet Jose • 3 years ago

Thanks. simple and crisp.

Reply • Share ›

SaltyCatFish • 3 years ago

Great post. Saving for reference.

Reply • Share ›

Ondra Medek • 3 years ago

Ping back: a logger per class: https://xmedeko.blogspot.cz...

Reply • Share ›

Colin Bitterfield • 3 years ago

Very nice write up. Very useful

Reply • Share ›

Ankush Sharma • 4 years ago

Thanks, I was using logging already but got some concepts cleared up !

Reply • Share ›

unknown_dev • 4 years ago

Amazing Article <3 love your explanation.

• Reply • Share ›

Neil F • 4 years ago

Is there a way that the log file is created ONLY if any write was made to it ? And if no one wrote to it the log file is not created at all?

• Reply • Share ›

Januka Samaranayake • 4 years ago

How do I send those loogers to logstash sever instead writing to a file

• Reply • Share ›

DeepFriedSeagull ➦ Januka Samaranayake • 4 years ago

Have a look at FileBeat -- https://www.elastic.co/prod...

• Reply • Share ›

Liang Du • 4 years ago

thanks !

• Reply • Share ›

caimaoy • 4 years ago

awesome

• Reply • Share ›

Amit Chaudhary • 4 years ago

Thanks for the explanation. Made it clear.

• Reply • Share ›

อนุกูล เทียนเกษมสุข (นุ) • 4 years ago

Thanks! It's nice article.

• Reply • Share ›

Ibrahim • 4 years ago

Thanks! excellent explanation.

But main.py module in its first appearance you have to add "import logging.config"
or you will get
AttributeError: module 'logging' has no attribute 'config'

• Reply • Share ›